# 🚀 DevOps CI/CD Pipeline (Jenkins + Docker + AWS EKS)

## 📄 One-line Summary

Containerized a static web application, pushed it to DockerHub, and implemented a Jenkins CI/CD pipeline to automatically build, push, and deploy the application to AWS EKS, making it accessible via a Kubernetes LoadBalancer.

## 📓 Brief Notes

### 1. Run the Application Locally

- Cloned GitHub repository
- Verified application using local server
- Confirmed static `dist/` files working

### 2. Dockerize the Application

- Created Dockerfile using Nginx
- Copied `dist/` files into container
- Built Docker image
- Ran container and verified on port

### 3. Push Image to DockerHub

- Logged into DockerHub
- Tagged Docker image
- Pushed image to repository

## 4. Setup Kubernetes (EKS)

- Created EKS cluster using `eksctl`
- Configured `kubectl`
- Verified cluster nodes

## 5. Deploy Application to EKS

- Created `deployment.yaml`
- Created `service.yaml`
- Applied using `kubectl`
- Retrieved LoadBalancer URL

## 6. CI/CD Pipeline Setup (Jenkins)

- Installed Jenkins on EC2 using Terraform
- Created pipeline job
- Integrated GitHub webhook
- Automated:
    - Docker build
    - Docker push
    - Kubernetes deployment

## 7. Monitoring & Logs

- Installed Prometheus & Grafana using Helm
- Monitored cluster and application
- Verified pod health and metrics

## 8. Submission Items

- GitHub repository
- Dockerfile
- Kubernetes YAML files
- Jenkinsfile
- README file
- Screenshots

# 🛠 Tools & Services Used

- GitHub – Source code management
- Jenkins – CI/CD automation
- Docker – Containerization
- DockerHub – Image registry
- Amazon EKS – Kubernetes cluster
- Kubectl – Kubernetes CLI
- AWS EC2 – Jenkins server
- Terraform – Infrastructure provisioning
- Prometheus & Grafana – Monitoring

# 🗂 Step-by-Step Conceptual Explanation

## 1) Source Control (GitHub)

**What it is:**
Central repository to store code

**Why it matters:**
Triggers CI/CD pipeline

**Flow:**
Push → Webhook → Jenkins

## 2) Local Verification

Ensures application works before deployment

## 3) Containerization (Docker)

**Concepts:**

- Image → blueprint
- Container → running instance

- Dockerfile → instructions

## 4) DockerHub

Stores Docker images for deployment

## 5) Kubernetes (EKS)

**Core Components:**

- Pod
- Deployment
- Service

## 6) Kubernetes YAML

Defines desired state of application

## 7) CI/CD Pipeline (Jenkins)

**Flow:**

1. Code push
2. Build image
3. Push image
4. Deploy to Kubernetes

## 8) Authentication & Permissions

- DockerHub credentials
- AWS access (IAM + kubeconfig)

## 9) Networking & LoadBalancer

- Exposes application to internet

- Used LoadBalancer service

## 10) Monitoring

- Grafana dashboards
- Prometheus metrics
- Jenkins logs

## 11) Deployment Strategy

- Rolling updates (zero downtime)

## 12) Security & Best Practices

- Use tokens instead of passwords
- Avoid hardcoding secrets
- Use IAM roles

## 13) Cost Awareness

- EKS cluster cost
- EC2 usage
- LoadBalancer charges

# 📃 Steps (Practical Execution)

## Step 1: Clone Project

```
git clone https://github.com/THeInfinityPro/Devops-Project-2.git
cd Devops-Project-2
```

## Step 2: Build Docker Image

```
docker build -t jagadishhak/project .
```

## Step 3: Push to DockerHub

```
docker login
docker push jagadishhak/project
```

## Step 4: Create EKS Cluster

```
eksctl create cluster --name Project --region us-east-1
```

## Step 5: Configure kubectl

```
aws eks update-kubeconfig --region us-east-1 --name Project
```

## Step 6: Deploy to Kubernetes

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

## Step 7: Get Application URL

```
kubectl get svc
```

## 📊 Monitoring Setup

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install monitoring prometheus-community/kube-prometheus-stack
```

Expose Grafana:

```
kubectl patch svc monitoring-grafana -p '{"spec": {"type": "LoadBalancer"}}'
```

## 🎯 End Result

- Application deployed on Kubernetes
- Accessible via LoadBalancer
- Fully automated CI/CD pipeline
- Monitoring enabled using Grafana

## 🧹 Cleanup

```
eksctl delete cluster --name Project --region us-east-1
terraform destroy
```

## 👨‍💻 Author

**Jagadish V**
DevOps Engineer (Fresher)
Chennai, India IN